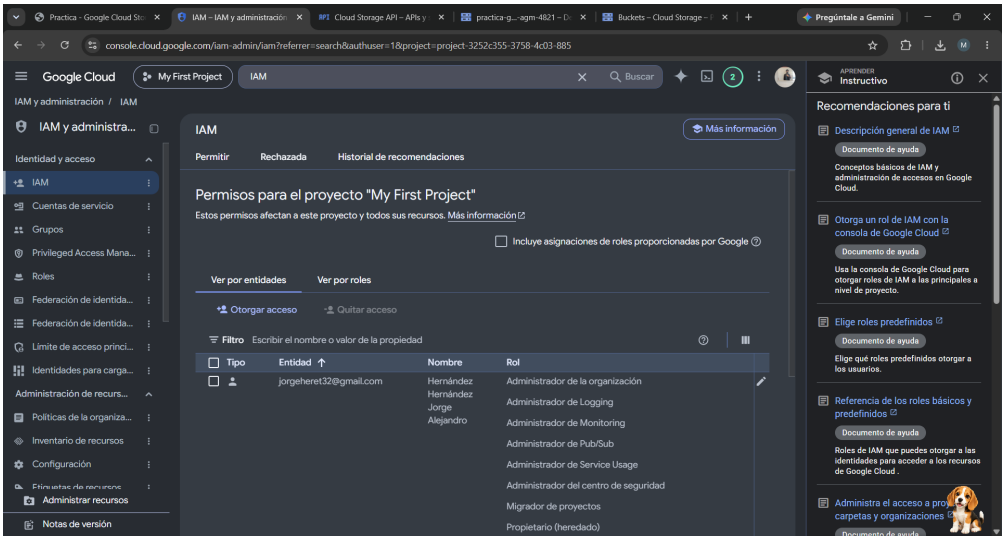
Google Cloud Storage JSON API

1. Seleccionar el proyecto

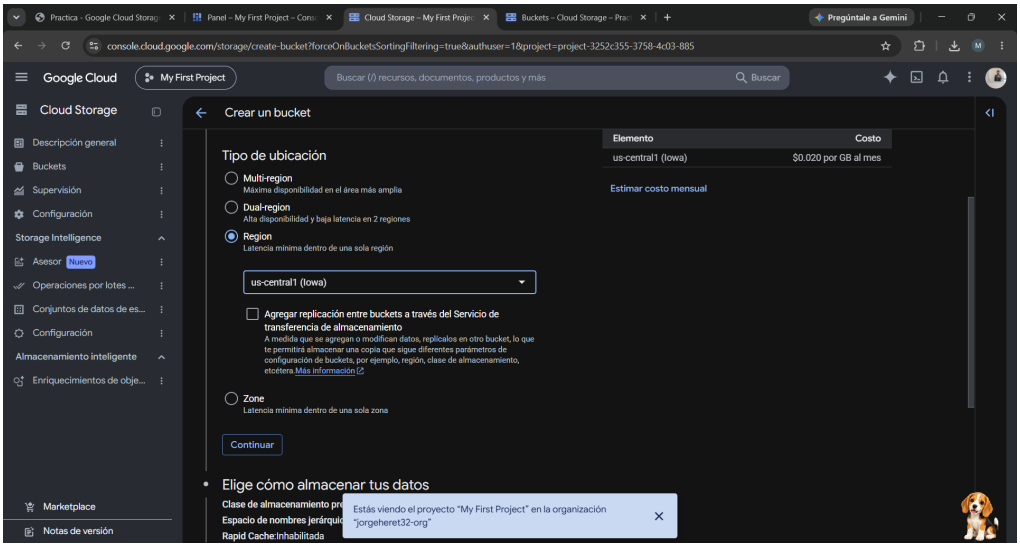
Se utilizó My First Project, el proyecto autorizado para la práctica. En la consola, el proyecto activo se muestra en el selector superior.



Proyecto activo en Google Cloud Console.

2. Crear el bucket

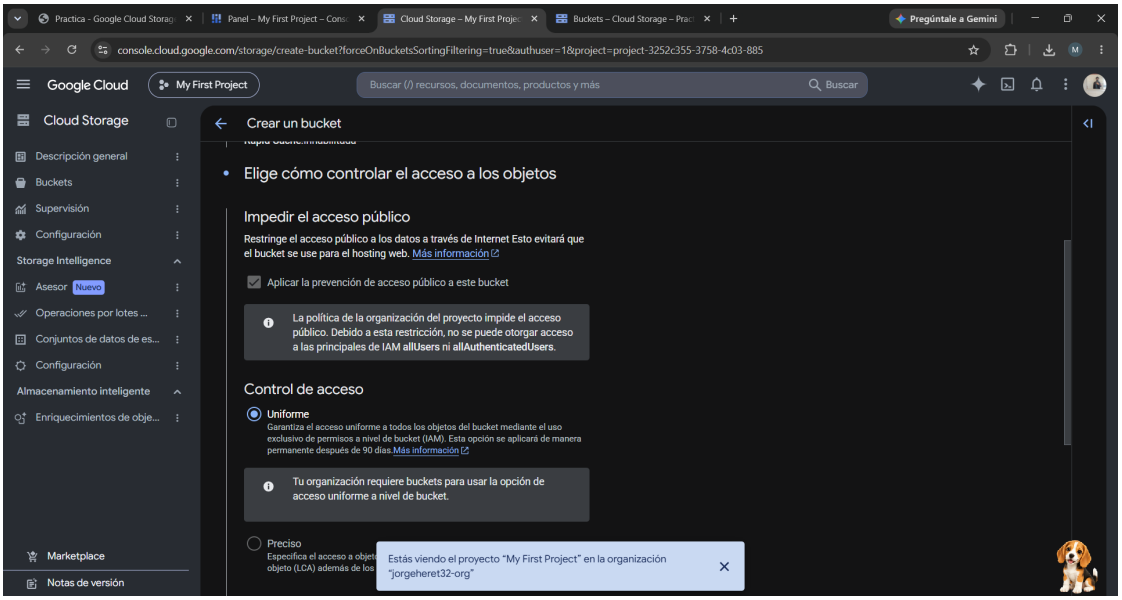
En Cloud Storage > Buckets > Crear se indicó practica-gcs-agm-4821 y se seleccionó Región, us-central1 (Iowa), y clase Standard.



Configuración de la ubicación del bucket.

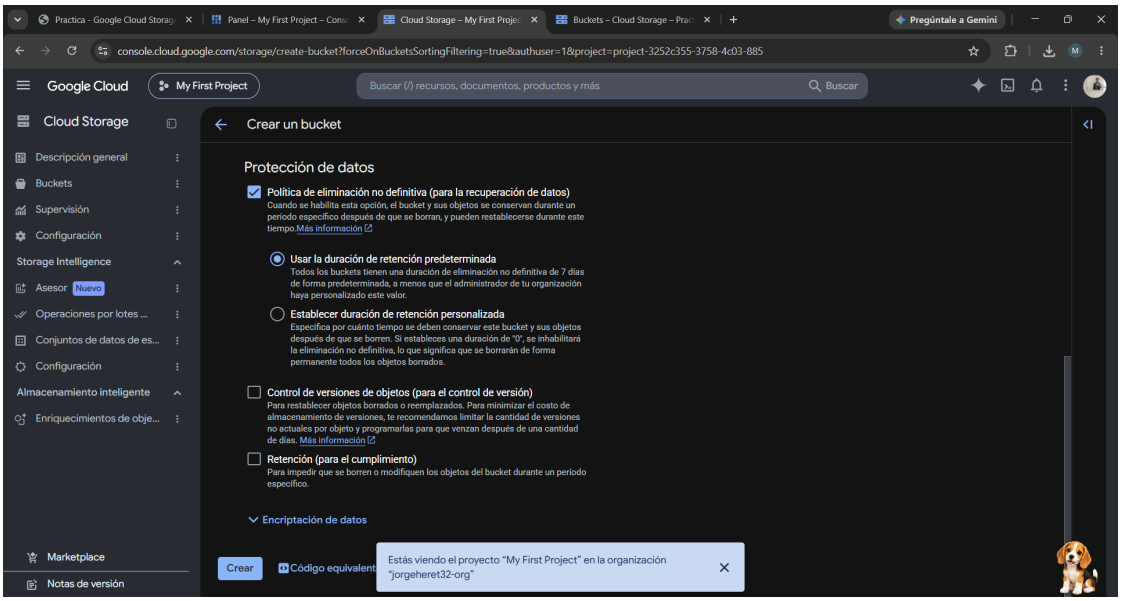
Acceso y protección de datos

En el paso 2 se mantuvieron activados la prevención de acceso público y el control Uniforme, requeridos por la organización del proyecto.



Prevención de acceso público y acceso uniforme.

Se mantuvo la duración predeterminada de eliminación no definitiva. Al final del formulario se pulsó Crear.

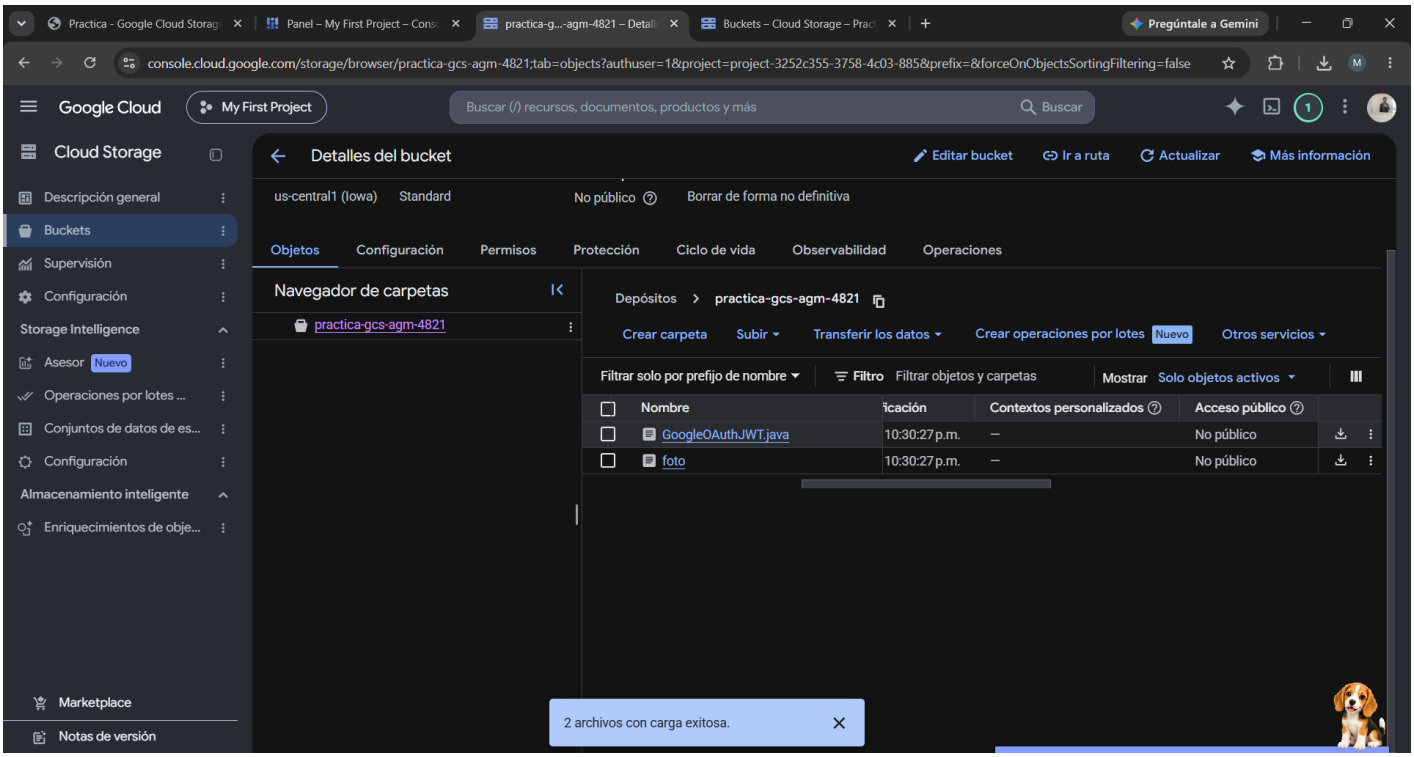


Opciones de protección de datos antes de crear el bucket.

Carga de los archivos de prueba

3. Subir el código y la imagen

En la pestaña Objetos se abrió el menú Subir y se cargaron GoogleOAuthJWT.java y la imagen de prueba de una suculenta. El nombre del objeto que se usó para descargarla fue foto.



Los dos objetos aparecen en el bucket con acceso no público.

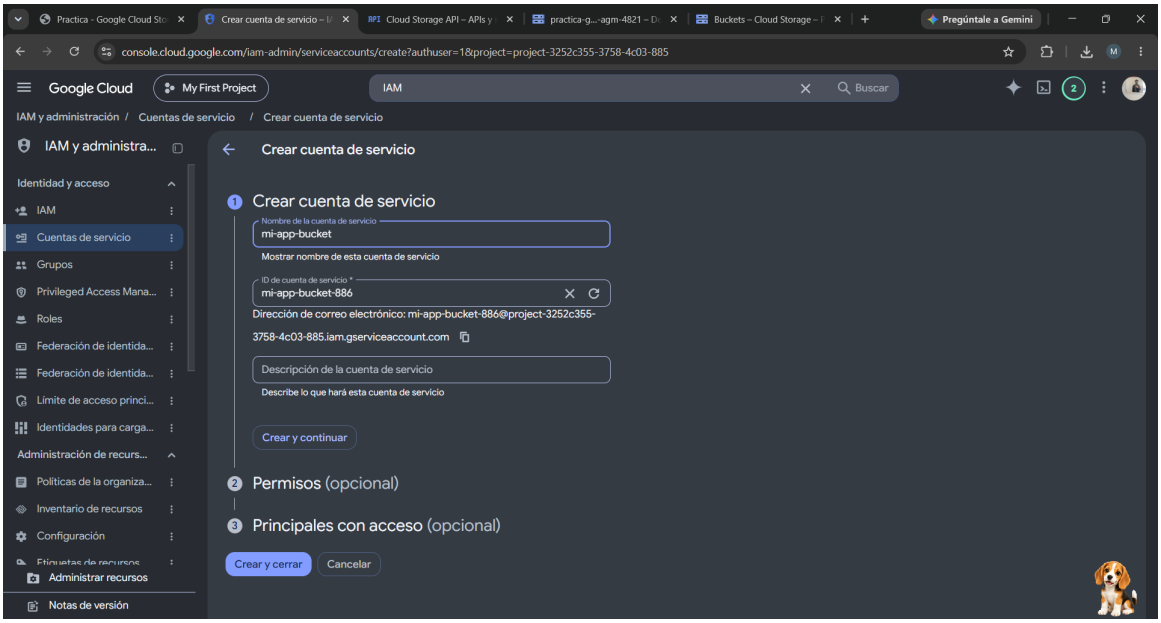
4. Verificar la API del proyecto

En APIs y servicios > Biblioteca se localizó Cloud Storage JSON API. Si la ficha muestra Habilitar, se activa desde ahí. La descarga posterior con curl confirmó que la API respondía.

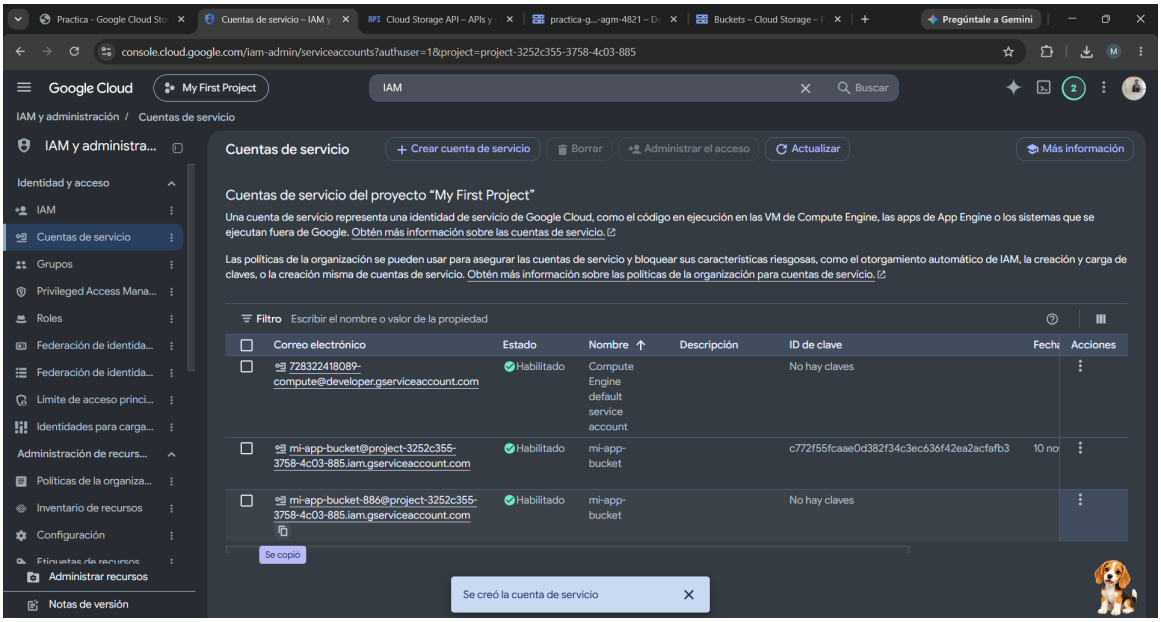
Cuenta de servicio

5. Crear la cuenta de servicio

En IAM y administración > Cuentas de servicio se creó mi-app-bucket con el ID mi-app-bucket-886. Se pulsó Crear y cerrar y se copió el correo de la nueva cuenta.



Formulario con el nombre y el ID de la cuenta.

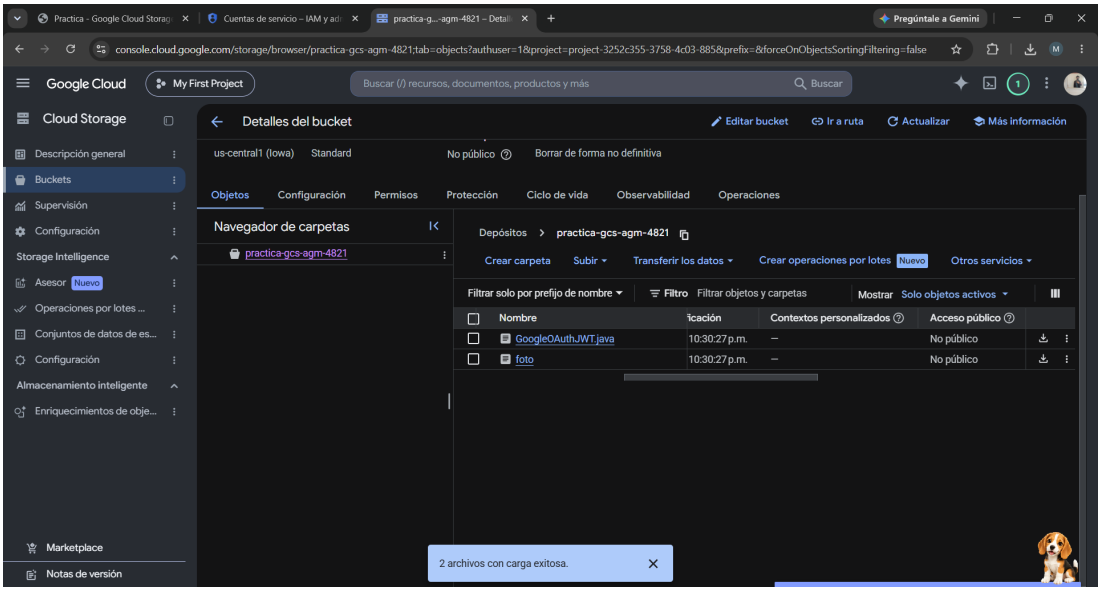


Confirmación de creación de la cuenta con sufijo -886.

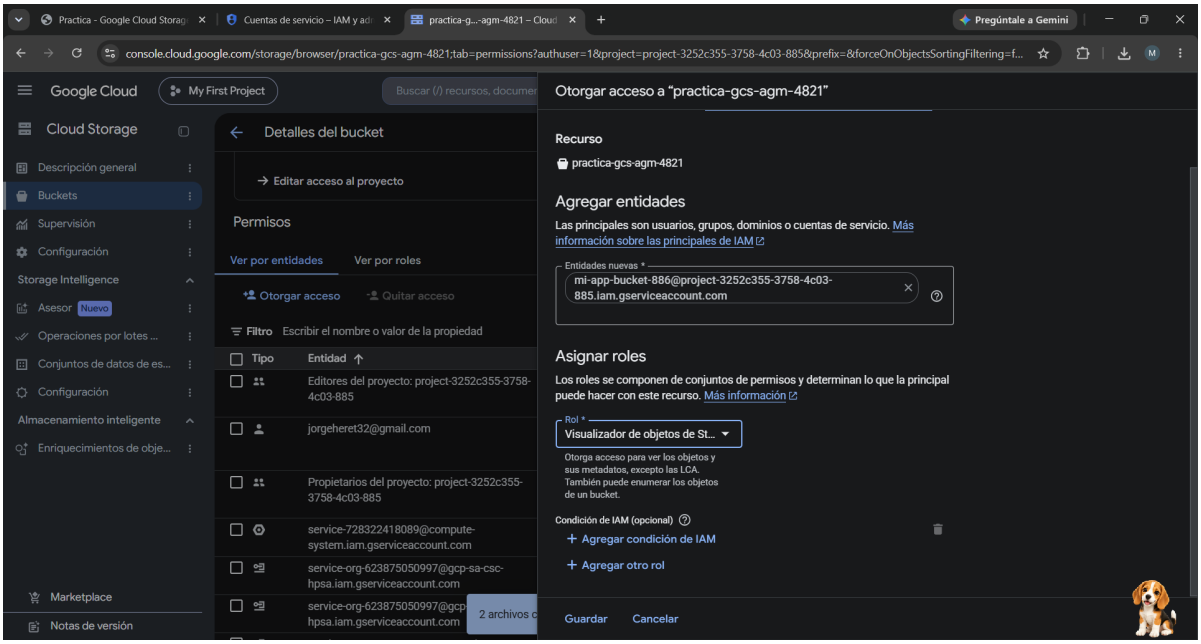
Acceso de lectura

6. Otorgar acceso a la cuenta

En el bucket practica-gcs-agm-4821 se abrió Permisos > Otorgar acceso. Se pegó el correo de mi-app-bucket-886 y se eligió Visualizador de objetos de Storage (roles/storage.objectViewer).



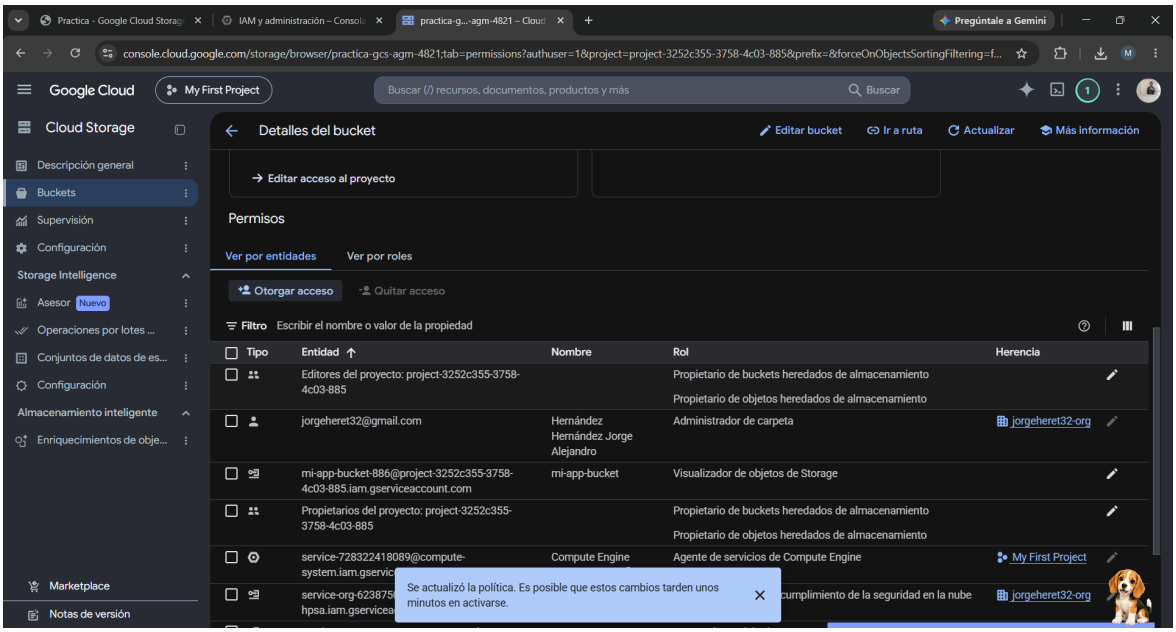
Pestaña Permisos en los detalles del bucket.



Asignación del rol de lectura a la cuenta de servicio.

Confirmación del permiso y clave JSON

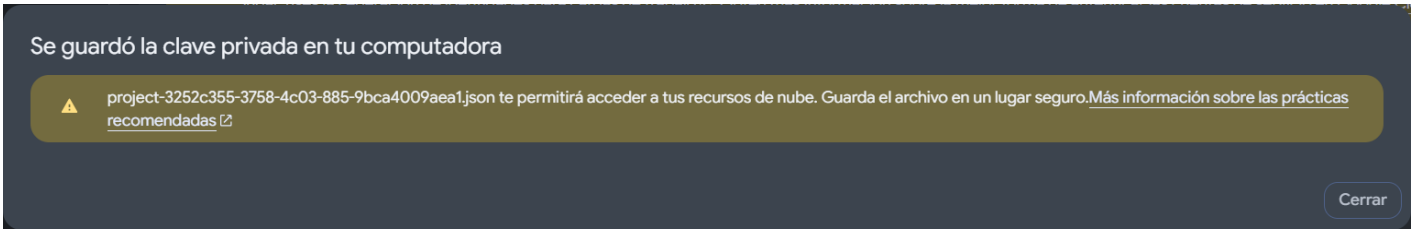
Al guardar el paso 6, la cuenta mi-app-bucket-886 quedó visible en la lista de permisos del bucket con el rol de lectura.



Confirmación del rol guardado en el bucket.

7. Descargar la credencial

En la cuenta de servicio se abrió Claves > Agregar clave > Crear clave nueva > JSON. La descarga se guardó en Descargas con el nombre credentials.json.

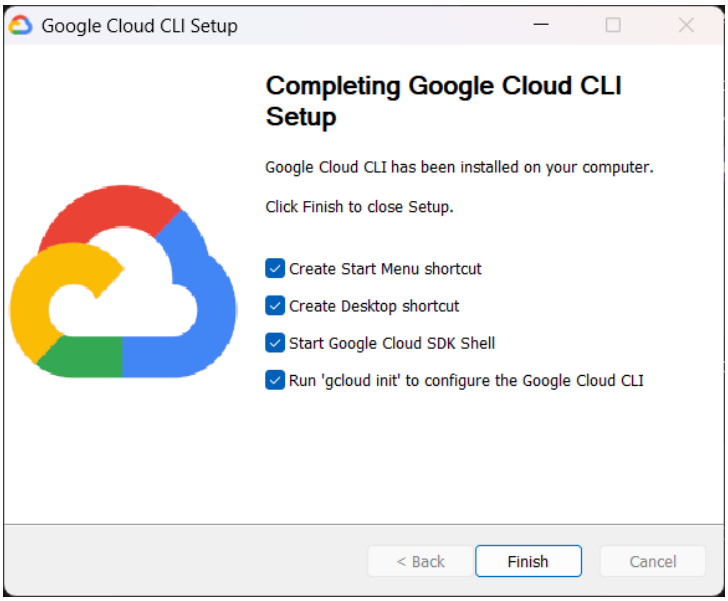


Confirmación de que la clave privada se guardó en el equipo.

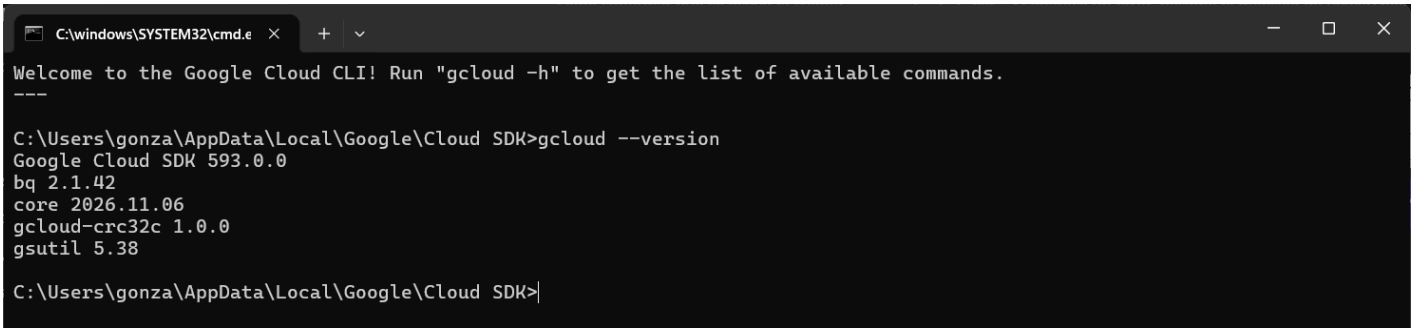
Google Cloud CLI en Windows

8. Instalar y verificar la CLI

Se instaló Google Cloud CLI para Windows. Al terminar se desmarcó Run gcloud init y se abrió Google Cloud SDK Shell. El comando gcloud --version confirmó la versión 593.0.0.



Pantalla final del instalador antes de desmarcar el inicio automático.



Comprobación de la instalación con gcloud --version.

PRUEBA MANUAL

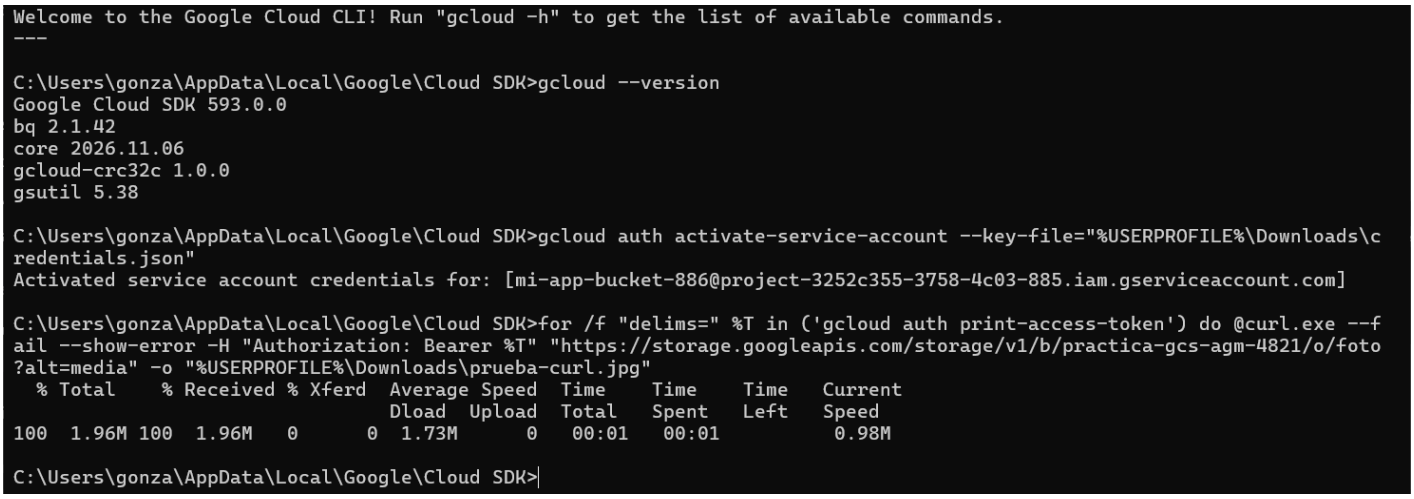
Descarga con curl

9. Activar la cuenta y descargar el objeto

En el Símbolo del sistema se activó la cuenta de servicio con la clave JSON. Después se descargó foto de la API JSON mediante una solicitud autenticada con el token de acceso.

```
gcloud auth activate-service-account ^
  --key-file="%USERPROFILE%\Downloads\credentials.json"

for /f "delims=" %T in ('gcloud auth print-access-token') do @curl.exe ^
  --fail --show-error -H "Authorization: Bearer %T" ^
  "https://storage.googleapis.com/storage/v1/b/practica-gcs-agm-4821/o/foto?alt=media" ^
  -o "%USERPROFILE%\Downloads\prueba-curl.jpg"
```



La activación se completó y curl recibió el 100 % del archivo.

Descarga desde Git Bash

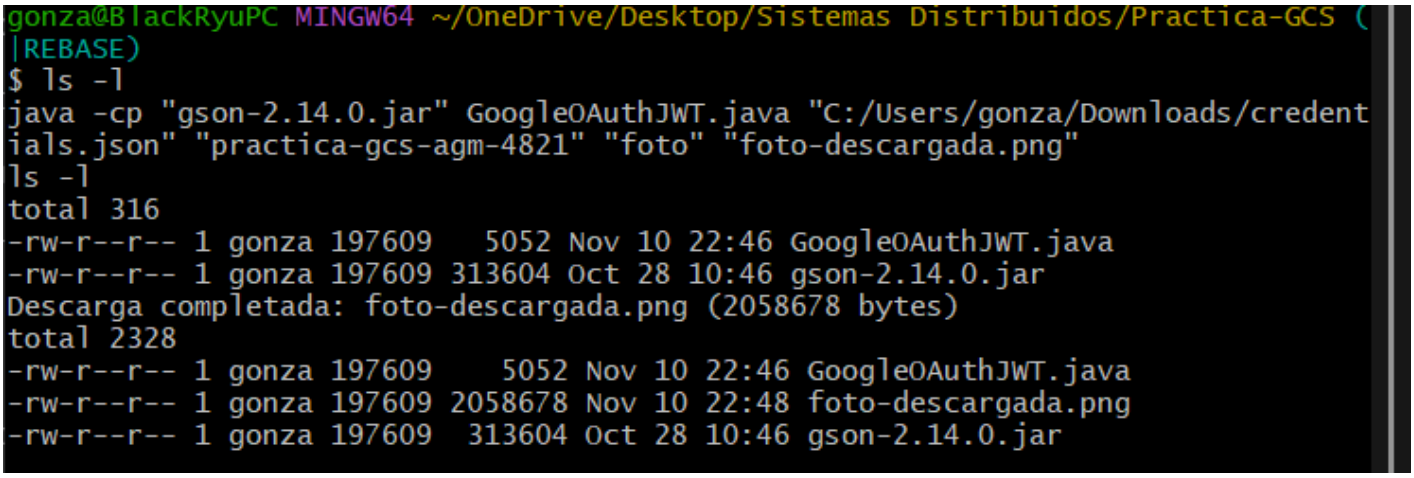
10. Completar el programa

El programa firma un JWT con RS256, lo intercambia por un token OAuth 2.0 y usa ese token en una petición GET con alt=media. Cuando la API devuelve HTTP 200, escribe los bytes del objeto en el archivo de salida. El código completo se incluye al final.

11. Ejecutar el programa

Con GoogleOAuthJWT.java y Gson 2.14.0 en Practica-GCS, se abrió Git Bash en esa carpeta. Se ejecutó el programa entre dos listados largos.

```
ls -l
java -cp "gson-2.14.0.jar" GoogleOAuthJWT.java \
  "C:/Users/gonza/Downloads/credentials.json" "practica-gcs-agm-4821" \
  "foto" "foto-descargada.png"
ls -l
```



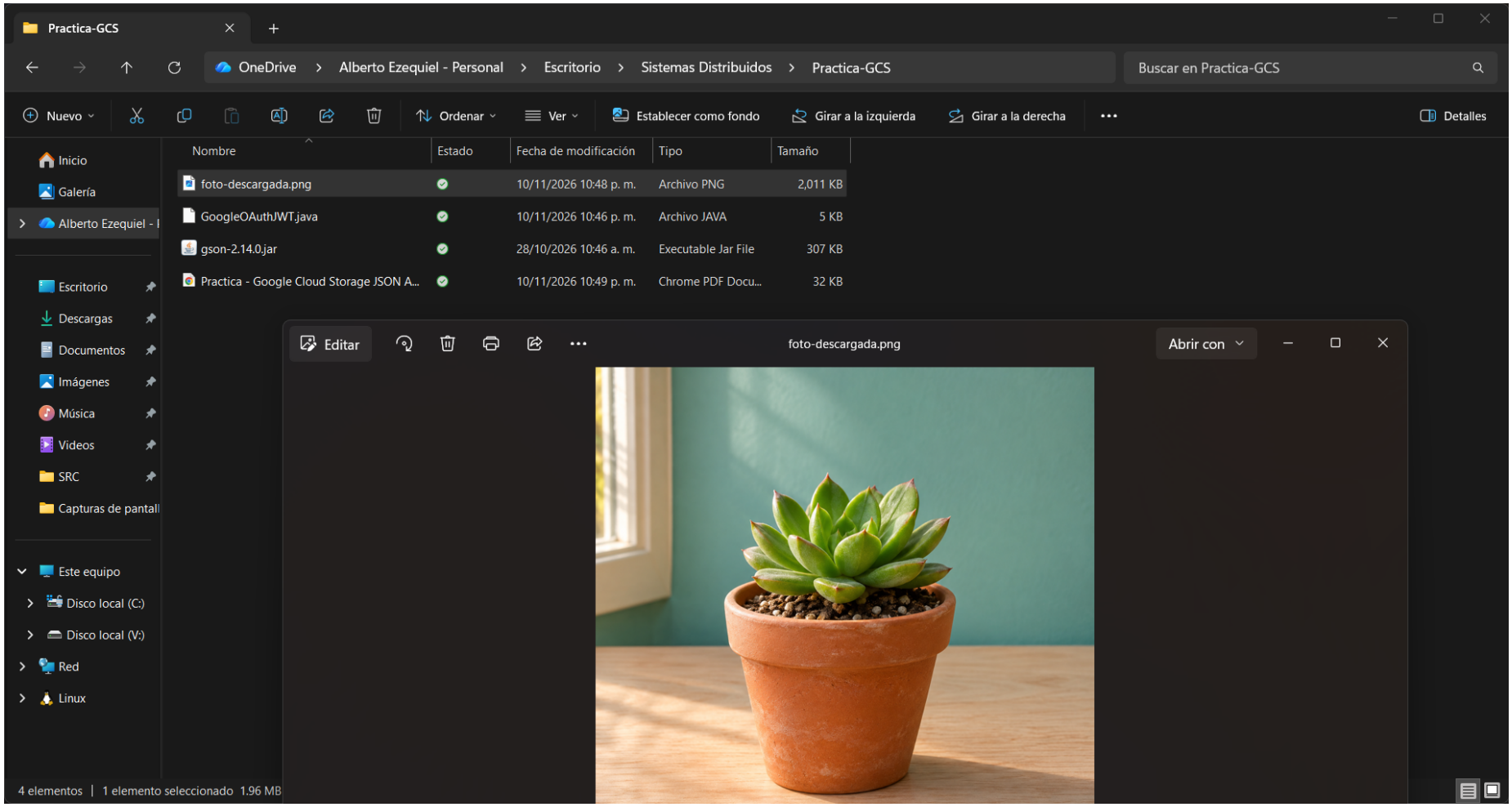
El listado final contiene foto-descargada.png con 2 058 678 bytes.

La descarga de Java y la de curl coincidieron byte por byte.

VERIFICACIÓN DEL PASO 11

Imagen descargada en Windows

Se abrió foto-descargada.png desde Practica-GCS y se comprobó que muestra la imagen de la suculenta.



El archivo descargado aparece en el Explorador y se abre en el visor de Windows.

CÓDIGO FUENTE

GoogleOAuthJWT.java

Líneas 1 a 66

```
import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.security.KeyFactory;
import java.security.PrivateKey;
import java.security.Signature;
import java.security.spec.PKCS8EncodedKeySpec;
import java.time.Instant;
import java.util.Base64;

import com.google.gson.JsonObject;
import com.google.gson.JsonParser;

public class GoogleOAuthJWT {

    public static void main(String[] args) throws Exception {

        // 1. Leer JSON credentials
        if (args.length != 4) {
            System.err.println(
                    "Uso: GoogleOAuthJWT <credentials.json> <bucket> <objeto> <archivo-local>");
            System.exit(1);
        }
        String json = Files.readString(Paths.get(args[0]), StandardCharsets.UTF_8);

        JsonObject creds = JsonParser.parseString(json).getAsJsonObject();

        String clientEmail = creds.get("client_email").getAsString();
        String privateKeyPem = creds.get("private_key").getAsString();
        String tokenUri = creds.get("token_uri").getAsString();

        // 2. Limpiar clave privada PEM
        privateKeyPem = privateKeyPem
                .replace("-----BEGIN PRIVATE KEY-----", "")
                .replace("-----END PRIVATE KEY-----", "")
                .replaceAll("\\s", "");

        byte[] keyBytes = Base64.getDecoder().decode(privateKeyPem);

        PKCS8EncodedKeySpec spec = new PKCS8EncodedKeySpec(keyBytes);
        KeyFactory kf = KeyFactory.getInstance("RSA");
        PrivateKey privateKey = kf.generatePrivate(spec);

        // 3. Crear Header JWT
        String headerJson = "{\"alg\":\"RS256\",\"typ\":\"JWT\"}";
        String header = base64UrlEncode(headerJson.getBytes(StandardCharsets.UTF_8));

        // 4. Crear Payload JWT
        long now = Instant.now().getEpochSecond();

        String payloadJson = "{"
                + "\"iss\":\"" + clientEmail + "\","
                + "\"scope\":\"https://www.googleapis.com/auth/devstorage.read_only\","
                + "\"aud\":\"" + tokenUri + "\","
                + "\"exp\":" + (now + 3600) + ","
                + "\"iat\":" + now
                + "}";

        String payload = base64UrlEncode(payloadJson.getBytes(StandardCharsets.UTF_8));

```

CÓDIGO FUENTE

GoogleOAuthJWT.java

Líneas 67 a 128

```
67          // 5. Firmar JWT
68          String unsignedToken = header + "." + payload;
69
70          Signature signature = Signature.getInstance("SHA256withRSA");
71          signature.initSign(privateKey);
72          signature.update(unsignedToken.getBytes(StandardCharsets.UTF_8));
73          byte[] signed = signature.sign();
74
75          String jwt = unsignedToken + "." + base64UrlEncode(signed);
76
77          // 6. Enviar a OAuth token endpoint
78          String requestBody = "grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer"
79                  + "&assertion=" + jwt;
80
81          HttpClient client = HttpClient.newHttpClient();
82
83          HttpRequest request = HttpRequest.newBuilder()
84                  .uri(URI.create(tokenUri))
85                  .header("Content-Type", "application/x-www-form-urlencoded")
86                  .POST(HttpRequest.BodyPublishers.ofString(requestBody))
87                  .build();
88
89          HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
90
91          // 7. Extraer el token de acceso
92          if (response.statusCode() != 200) {
93              throw new IllegalStateException("Error al solicitar el token OAuth. HTTP "
94                      + response.statusCode());
95          }
96          String accessToken = JsonParser.parseString(response.body())
97                  .getAsJsonObject()
98                  .get("access_token")
99                  .getAsString();
100
101         // 8. Descargar el contenido del objeto
102         String encodedObject = URLEncoder.encode(args[2], StandardCharsets.UTF_8)
103                 .replace("+", "%20");
104         URI downloadUri = URI.create("https://storage.googleapis.com/storage/v1/b/"
105                 + args[1] + "/o/" + encodedObject + "?alt=media");
106
107         HttpRequest downloadRequest = HttpRequest.newBuilder(downloadUri)
108                 .header("Authorization", "Bearer " + accessToken)
109                 .GET()
110                 .build();
111         HttpResponse<byte[]> download = client.send(
112                 downloadRequest, HttpResponse.BodyHandlers.ofByteArray());
113
114         if (download.statusCode() != 200) {
115             throw new IllegalStateException("Error al descargar el objeto. HTTP "
116                     + download.statusCode());
117         }
118
119         // 9. Guardar el archivo y mostrar el resultado
120         Files.write(Paths.get(args[3]), download.body(), StandardOpenOption.CREATE_NEW);
121         System.out.printf("Descarga completada: %s (%d bytes)%n",
122                 args[3], download.body().length);
123     }
124
125     private static String base64UrlEncode(byte[] input) {
126         return Base64.getUrlEncoder().withoutPadding().encodeToString(input);
127     }
128 }
```